

5월 27일 최종 발표 storyline (v2, 4차 정정)

팀: 속도는벡터 (박세은·강재현·조현빈·이동욱) 발표 일시: 2026-05-27 (화) 19:00 분량: 약 15분 (슬라이드 20장 안팎) + Q&A 작성: 2026-05-14 (5/13 v1 → 5/14 v2 4차 정정)

들어가는 말

이 글은 5월 27일 최종 발표를 어떤 순서로, 어떤 흐름을 가지고 진행할지 정리한 발표용 시나리오다. 4월 28일 중간 보고서에서 정리한 RQ1·RQ2 결과를 바탕으로 5월 한 달 동안 진행한 RQ3 측정과 추가 분석을 발표 자리에서 자연스럽게 풀어 가는 데 초점을 두었다. 슬라이드 한 장 한 장의 문구가 아니라, "왜 이런 흐름이 필요한가"를 우리 팀 안에서 합의하기 위한 글이다. 슬라이드는 이 문서를 기준으로 추후 정리한다.

본 v2의 핵심 변화 두 가지를 먼저 짚어 둔다. 첫째, 5월 14일 새벽까지 추가 측정 (가중치 변화 sweep + 저비용 결합 방식 4 후보 + 자원 효율 Pareto 분석)이 회수되면서 narrative가 확정되었다 — **단독 대체 가능 method의 발견이 본 연구 main finding 이고, 결합 framework의 가치는 "더 큰 개선"이 아니라 "안정성 보강 + 자원 효율"이라는 정직한 narrative**. 둘째, 박세은 5월 13일 12:13 카톡 피드백 (method 개수 너무 많음 + 숫자/공식 최소화)을 반영해서 본문에서는 핵심 method 5개만 자세히 짚고, 폐기 method명 전체 list와 통계 검정 jargon은 부록으로 분리했다.

발표 전체 흐름은 청자가 자연스럽게 따라올 수 있는 일곱 단계의 순차 흐름으로 잡았다. 먼저 우리가 들여다본 문제를 정의하고, 분포 인지 방법 56개를 탐색하면서 어떤 것이 실제로 측정 가능한지를 정리한 다음, 단독 대체 가능성을 분석하고, 그 결과를 바탕으로 결합 framework 검토로 넘어가고, 자원 효율을 함께 살펴본 뒤, 마지막에 성능과 자원 사이의 경계에서 본 연구의 권장 design을 추출하고, 발표를 마무리한다.

1. 문제 정의 — 우리가 들여다본 영역 (약 2분)

발표를 시작하면서 가장 먼저 청자에게 전달하고 싶은 것은 우리가 푼 문제의 위치다. 최근 데이터베이스 환경에서는 이미지나 텍스트를 임베딩 벡터로 바꿔서 검색하는 작업과, 날짜·가격 같은 일반적인 표 데이터를 SQL로 분석하는 작업이 한 데이터베이스 안에서 동시에 일어난다. 이걸 벡터 증강 분석 쿼리 (VAQ, Vector-augmented Analytical Query)라고 부른다. 우리 연구는 이런 쿼리를 잘 처리하려면 옵티마이저가 결과 행 수를 얼마나 정확히 추정하느냐, 즉 카디널리티 추정이 핵심이라는 점에서 시작한다.

문제는, pgvector, VBASE, DuckDB 같은 기존 시스템들이 벡터 조건의 선택도를 33.3%, 50%, 100% 같은 고정 비율로 둔다는 것이다. 실제 선택도는 데이터와 쿼리에 따라 0.001% 부터 100% 까지 크게 바뀐다. 잘못된 선택도 추정은 잘못된 실행 계획으로 이어지고, 잘못된 실행 계획은 최대 1만 배까지 느려지는 결과를 낳는다. 이 문제를 해결하기 위해 Exqutor 라는 논문이 두 가지 방법을 제시한다. 인덱스가 있을 때는 HNSW range query 로 실제 카디널리티를 구하고, 인덱스가 없을 때는 적응적 샘플링 (Adaptive Sampling) 으로 추정한다.

우리 연구는 이 중에서 두 번째, 즉 인덱스가 없을 때 작동하는 적응적 샘플링 영역을 다룬다. 본 논문은 이 부분에서 베르누이 무작위 샘플링을 사용한다. 우리의 질문은 단순하다 — 데이터가 한쪽에 몰려 있을 때 (skew 분포), 이 무작위 샘플링이 충분히 정확한가? 그리고 더 정확하게 만들 수 있는 방법은 무엇인가? 이 출발점에서 RQ1 과 RQ2 가 자연스럽게 따라온다.

RQ1 (약 1분 내 압축): PostgreSQL 의 두 표본 추출 방식 (SYSTEM 블록 단위 vs BERN 행 단위) 을 짝지어 비교한 결과, 모든 선택도 구간에서 SYSTEM 이 BERN 보다 부정확하며, SIFT 의 선택도 0.05 구간에서 최대 17.32% 격차다. 본 논문이 사용한 베르누이가 이미 PostgreSQL 두 옵션 중 더 나은 쪽이지만, 베르누이조차 분포에 따라 정확도가 달라진다는 출발점이 된다.

RQ2 (약 1분 내 압축): 분포를 미리 안다는 가정 아래, 다섯 가지 할당 방식 (베르누이 / 동등 / 비례 / Neyman / 반 (反) Neyman) 의 짝지어 비교 결과, "베르누이에서 비례 할당으로 바꾸면 평균 9.53% 정확도 개선" 이 한 줄 요약이다. Neyman 이 비례보다 약간 부정확한 역설 (1.595 vs 1.580) 은 PartSupp PK 의 클러스터 균등 분포 (변동 계수 = 0) 와 좁은 분산 범위 (1.3~1.6 배) 가 만든 자연스러운 결과로, 분포를 알면 비례 할당이 답이지만 실제 운영에서는 분포를 모를 때가 많다는 점이 RQ3 로 이어지는 출발점이다.

2. 분포 인지 방법 56 개의 탐색 — 어떤 것이 측정 가능하고 어떤 것이 폐기되는가 (약 2.5분)

RQ3 의 첫 단계는 분포 정보를 얻으려는 후보 method 56 개를 8 패러다임 (클러스터링, 공간 분할, 스트리밍, 차원 축소, 정보 이론, 양자화, 준 무작위, 밀도 추정) 으로 모아 9 cell 매트릭스에서 측정을 시도한 것이다. 측정을 진행하면서 일부 method 는 그대로 둘 수 없는 문제가 발견되어 폐기했는데, 그 사유를 3 범주로 정리한다.

2.1 폐기 사유 3 범주

(1) **자원 한계** — 측정 서버의 메모리 한계로 실행이 불가능하거나, 한 cell 측정에 4 시간 이상의 timeout 이 필요해서 전체 portfolio 마감을 맞추기 어려웠던 method. 가장 인상적인 birch 는 클러스터를 트리 구조로 저장하는 부분이 50 ~ 200GB 메모리를 차지해서 실행 자체가 불가능했고, agglomerative 는 256 차원 데이터셋에서 모든 점 쌍의 거리를 한꺼번에 계산하다가 메모리 부족으로 실패했다. kde_parzen 은 한 cell 측정에 4 시간 이상의 timeout 이 필요해서 전체 portfolio 마감 안에 완성할 수 없었다.

(2) 알고리즘 구현 결함 — 5월 10일에 8-agent 정독 검토로 발견된 reference 위반이나 알고리즘 잘못 표기. 가장 인상적인 사례는 vinecopula 의 코드 구현이 rank 변환 + PCA 1 차원 정렬의 별칭으로 되어 있어서 Bedford-Cooke 1986 의 진짜 vine copula 가 아니라는 점, 그리고 neuram 이 코드 한 줄씩 검토한 결과 PCA1D 와 100% 동일하다는 점이다. kdtree 의 알고리즘 구현 코드를 보면 leaf 인덱스가 단순한 modular hash (`idx % n_strata`) 로 처리되어 있어서 무작위 표집과 거의 동등 — paper 가 의도한 kdtree 의 공간 분할 효과가 아니다.

(3) 적합성 위반 — 큰 데이터셋에서 추정값이 +145,483%, +213,065% 같은 외곽 값으로 나타나서 paper 가 정의한 sample budget 안에서 estimator 의 적합성을 보장하지 못하는 method.

이 과정을 거치고 남은 측정 가능한 method 는 약 43 개다. 이들이 본 연구의 비교 실험군이며, 8 패러다임 분류 위에서 측정 결과를 정리한다. **폐기 method 명 전체 list 는 부록 H 로 분리** (박세은 5/13 12:13 피드백 반영).

2.2 남은 43 method 의 paradigm 별 평균 결과 — 어떤 패러다임이 좋은가

남은 method 의 결과를 8 패러다임으로 묶어 평균 효과 (결합 모드 기준) 를 보면 다음과 같다.

paradigm	평균 개선폭	대표 anchor method
밀도 추정	-11.93% (n=1)	Parzen KDE
정보 이론	-7.60% (n=9)	HyperLogLog
스트리밍	-6.63%	Chao 1982 weighted reservoir
차원 축소	-6.03%	Sparse Random Projection
공간 분할	-5.57%	Hilbert curve, Z-order

이 표만 보면 "어떤 패러다임이 가장 좋은가" 에 대한 답이 보이지만, paradigm 평균이 일부 method 의 극단값에 끌려가는 경우가 있다 (예: 클러스터링의 wavelet_hist 가 +68%). 그래서 본 연구는 paradigm 평균을 보조 정보로 두고, 다음 단계에서 단독 대체 가능성을 method 단위로 살펴본다.

3. 단독 대체 가능성 분석 — paper 의 베르누이를 우리 method 로 바뀔 도 되는가 (★ 본 연구 main finding, 약 2.5분)

다음 질문은 자연스럽다. 남은 43 method 의 추정값을 본 논문의 베르누이 대신 그대로 갈아 끼우면 (CaseA 단독 대체 모드) 어떻게 되는가?

측정 결과를 정직하게 풀어 정리하면 다음과 같다.

- **평균 우위 약 40%:** 56 방법 중 약 40% 가 평균적으로는 베르누이 기준선보다 정확했다.

- **통계 일관 우위 7.6%:** 그러나 같은 method 라도 데이터셋과 cell 마다 효과의 편차가 컸다. 통계 검정에서 cell 전반에서 안정적으로 베르누이를 우위로 누른 method 는 7.6% 정도다.
- **★ 15 method 의 평균 개선폭 -5 ~ -12%:** 단독 대체로 통계 일관 우위인 15 method 의 평균 개선폭은 -5 ~ -12% 다. 이는 paper 자체 재현 시 발생하는 측정 변동 (-4.3%) 보다 1.2 ~ 3 배 큰 의미 있는 개선이다. 즉 단독 대체로도 noise floor 를 넘는 실제 개선이 가능한 method 가 존재.
- **단독 best:** 본 측정 portfolio 의 단독 best 는 minibatch_partial 의 -10.17% 개선이며, 결합 best (-7.37%) 보다도 큰 개선폭이다.

3.1 단독 대체 가능 method 5-6 후보 (5/14 정리)

핵심 method 5-6 개의 알고리즘 메커니즘과 자원 효율을 자세히 소개한다 (박세은 5/13 12:13 피드백 — method 개수 줄임 + 핵심만 깊이 소개).

sparse_rp (Sparse Random Projection, Li-Hastie-Church 2006). 차원 축소 paradigm. 고차원 벡터를 무작위 sparse projection 행렬로 저차원에 사영. 학습 0.1초로 본 연구 portfolio 의 가장 cheap 영역. 평균 -5 ~ -12% 개선.

chao_weighted (Chao 1982 weighted reservoir sampling). 스트리밍 paradigm. 확률에 가중치를 곱한 reservoir 표본을 streaming 으로 유지. 메모리 매우 적음 (O(K), KB 영역) + 정확도 anchor 수준.

minibatch_partial (mini-batch K-means, Sculley 2010 chunk-only). 클러스터링 paradigm. chunk 단위로 K-means centroid update. 본 측정 portfolio 의 단독 best (-10.17%).

hilbert_real (진짜 Hilbert curve, Wikipedia 표준). 공간 분할 paradigm. space-filling curve 의 spatial locality 로 stratum 분할. 학습 cheap + 정확도 강력.

hyperloglog (Flajolet-Fusy-Gandouet-Meunier 2007). 정보 이론 paradigm. cardinality 추정의 sketch 자료 구조. paper Eq 1-6 의 모멘텀 update 와 매우 결합 친화.

★ **reservoir** (Vitter 1985 simple reservoir). 스트리밍 paradigm. **메모리 O(1) — 본 측정 portfolio 최저** + 학습 0.1초 + 정확도 anchor 수준 (-9.25%). 산업 적용 영역에서 가장 강력한 finding (5 장에서 다룸).

다만 단독 대체는 cell 별 spread 가 커서 산업 적용 보장에는 안정성 부족 — 이 안정성 부족이 다음 단계 (결합 framework 검토) 의 motivation 이다.

4. 결합 framework 검토 — 산술 평균이 효과적인가 (약 2분)

단독 대체의 안정성 부족을 보완하기 위해 다음으로 검토한 방향이 두 추정값을 결합하는 framework 다. 처음에는 가장 단순한 결합 방식 — 산술 평균 (0.5 / 0.5) — 으로 측정을 진행했다.

결합 모드의 측정 결과: 베르누이 추정값과 우리 method 추정값을 단순 평균한 결과, 492 개의 짝지어 비교한 측정 중 92.5% 에서 베르누이 단일 baseline 보다 더 정확하다. 통계 검정에서도 분명한 차이 (p-value 매우 작은 값). 12 anchor method 는 cell 전반에서 -9 ~ -10% 의 일관된 개선과 안정적인 spread 를 보인다.

왜 산술 평균이 효과적인가: 베르누이 무작위 샘플링은 편향이 0 이지만 분산이 크다. 클러스터 기반 계층적 표집 추정량은 분포 가정이 맞을 때는 분산이 작지만, 가정이 빗나가면 편향이 생길 수 있다. 두 추정값을 산술 평균하면 한 쪽이 실패할 때 다른 쪽이 보완해 주는 안정적인 구조가 된다.

4.1 가중치 sweep 결과 (5/14 새벽 회수)

산술 평균 외 다른 가중치도 측정해 봤다. 베르누이 가중치 0.3 / 0.4 / 0.5 / 0.6 / 0.7 다섯 값으로 가중 평균을 측정한 결과:

- 4 anchor method 중 3 개가 가중치 0.5 (산술 평균) 에서 best.
- 양쪽 극단 (0.3 / 0.7) 에서 효과 감소.

→ 산술 평균이 결합 방식 중 best 이며, 가중치 변화로 더 큰 개선 어렵다.

4.2 저비용 결합 방식 4 후보 결과 (5/14 새벽 회수)

산술 평균 외 다른 결합 방식 후보 — Centroid tuple / Hash bucketing / PCA preprocessing / Iterative refinement — 의 32 measurement 회수 결과:

- Centroid tuple 만 결합 모드 4 method 모두에서 보편 우위 (평균 -0.84%p 추가 정확도). 학습 비용 추가 0 + 더 좋은 정확도의 "더 싸고 더 좋은" 패턴.
- 나머지 3 후보 (Hash / PCA / Iterative) 는 method × mode 별로 spread 크거나 일부 영역에서 marginal/harmful.

4.3 결합 framework 의 진짜 위치 — 정직 disclosure

가중치 sweep + 4 cheap 근사 후보 측정을 종합한 결과:

- 단독 best (-10.17% minibatch_partial) > 결합 best (-7.37% sparse_rp Centroid tuple)
- 결합으로 단독 best 능가 불가 — 이것이 본 연구의 정직한 disclosure
- 결합의 진짜 가치 = method 선택 robustness + cell spread 줄임 ("더 큰 개선" 이 아님)

발표 자리에서는 "결합으로 단독 best 능가 X 는 정직한 finding 이며, 결합의 진짜 가치는 안정성 보강" 이라고 짚는다.

5. 자원 효율 분석 — 학습 비용을 고려하면 어떤 method 가 실용적인가 (★ 산업 적용 핵심, 약 2.5분)

성능만으로 산업 적용을 결정할 수 없다. 학습 시간, 메모리 사용, 차원 한계가 method 마다 다르기 때문이다.

5.1 Pareto frontier Top 5

학습 시간과 정확도 개선폭의 두 axis 로 Pareto frontier 를 도출하면 다음 5 method 가 frontier 상에 위치한다.

Method	학습 시간	정확도 개선	메모리
sparse_rp	0.1초	-9.43%	$O(D \cdot k)$ ~MB
chao_weighted	0.5초	-9.60%	$O(K)$ ~KB (최저 영역)
neuram	0.5초	-9.97% (최고)	$O(K \cdot D)$ bounded
pca1d	0.5초	-9.63%	$O(N)$
hilbert / hilbert_real	0.1-0.5초	-9.27 ~ -9.41%	$O(N)$

이 5 method 는 12 anchor 일관성 명단과도 일치한다. 학습 시간 0.1-0.5 초의 매우 적은 비용으로 anchor 수준 정확도 개선을 제공한다.

5.2 산업 적용 3 영역 추천 — ★ reservoir $O(1)$ finding

영역 A — Best of Both Worlds (일반 OLAP server): sparse_rp 또는 chao_weighted. 학습 0.1-0.5초 cheap + 정확도 anchor 수준 + 메모리 작음. 본 연구의 가장 균형 잡힌 권장.

영역 B — Quality-First (정확도 우선): neuram. 학습 0.5초 이지만 본 측정 portfolio 정확도 최고.

★ 영역 C — Resource-First (모바일/embedded/streaming): reservoir. 학습 0.1초 + 정확도 anchor 수준 + 메모리 $O(1)$ — 본 측정 portfolio 최저. 자원 제약이 매우 큰 환경 (메모리 < 100MB, 학습 시간 < 100ms) 에 가장 적합. 이 발견이 본 연구의 가장 강력한 산업 적용 finding 이다.

5.3 cheap 근사 — Centroid tuple 의 "더 싸고 더 좋은" 패턴

multi-table cell (예: DEEP + WIKI cross) 에서 계층적 표집을 어떻게 할지의 문제가 있다. 두 가지 후보가 있다.

- **비싼 multi-join 재학습:** 두 벡터를 864 차원으로 합친 후 KM20 을 처음부터 다시 학습. 학습 시간이 single-table 의 2 배 + 추가 disk I/O.
- **cheap 근사 (Centroid tuple):** 두 single-table 의 클러스터 결과를 tuple 로 합쳐서 그대로 사용. 학습 비용 0 추가.

8 measurement 결과를 비교하면, 결합 모드에서 4 method 모두 cheap 근사가 비싼 재학습보다 평균 -0.84%p 정확도가 더 좋다. "더 싸고 더 좋은" best of both worlds 결과로, multi-table 영역 확장의 cheap 근사 가능성을 보여 준다.

5.4 단독 대체 vs 결합의 자원 비교

단독 대체는 우리 method 만 사용하므로 학습 비용 = 우리 method 학습 비용. 결합은 베르누이와 우리 method 두 estimator 의 합인데, paper 의 sample budget 을 두 estimator 가 공유하므로 query 시 추가 시간은 거의 없다. 학습 단계에서도 우리 method 만 학습하면 되므로 단독 대체와 동일. 즉 결합의 자원 추가 부담은 무시 가능.

6. 성능과 자원 사이의 경계 — 본 연구의 권장 design (약 1.5분)

성능 (단독 대체 -5 ~ -12% + 결합 92.5% 안정성) 과 자원 (cheap 근사로 0 학습 비용 추가) 사이의 균형 영역에서 본 연구가 권장하는 design 을 추출하면 다음 세 원칙의 조합이다. 본 v2 의 핵심 narrative 갱신은 단독 대체 우선 + 결합 보조의 흐름이다.

1. **단독 대체 적용 우선** — 단독 대체 가능 5-6 method (sparse_rp / chao_weighted / minibatch_partial / hilbert_real / hyperloglog / reservoir) 중 산업 환경에 맞는 method 선택 후 paper 베르누이를 우리 method 의 추정값으로 같이 끼우는 가장 단순한 방식. 평균 -5 ~ -12% 의 의미 있는 정확도 개선 가능.
2. **결합 framework 보조 사용** — 단독 대체의 cell 별 spread 가 산업 적용에 부담될 때, 산술 평균 결합으로 안정성 보강. 92.5% 찍지어 비교 우위로 cell spread 줄어 들고 method 선택 robustness 확보. multi-table 영역에서는 Centroid tuple 저비용 결합 방식이 학습 비용 추가 0 으로 추가 정확도 개선 제공.
3. **method-aware 선택적 적용** — 12 anchor method 중에서도 cell 별 K-민감도와 quality-민감도 패턴이 다르므로, sparse_rp 와 chao_weighted 에는 정밀 처리 (K=20 + multi-join 재학습), Hilbert curve 와 HyperLogLog 에는 단순 처리 (cheap 근사 + carry-over) 를 선택적으로 적용.

산업 적용 영역은 두 가지다. (1) PostgreSQL pgvector 의 default sampling 메커니즘에 우리 method 의 stratified estimator 를 단독 대체 또는 산술 평균 결합으로 추가. (2) Exqutor 의 \$V-B 모듈에 우리 안 (단독 대체 우선 + 결합 보조 + cheap 근사 + method-aware) 을 통합하면 paper 본문 성과 (1만 배 속도 개선) 위에 추가적인 정확도 layer 를 얹을 수 있다.

본 연구는 또한 추가 가설 두 가지 — multi-table 재계층화의 정확도 영향과 저비용 근사 가능성 — 도 정량 검증하였다. 측정 결과 method 가 환경 변화에 보이는 민감도 패턴이 method 의 내부 메커니즘 차이를 반영하는 부수 발견도 정리하였다 (자세한 내용은 부록 참조).

7. 마무리 — 한계와 향후 연구 (약 1.5분)

발표의 마지막은 우리 연구의 한계를 정직하게 짚고, 6월 11일 최종 보고서에서 어떤 부분을 추가로 정리할지를 공유한다.

가장 중요한 한계는 측정 범위다. 9 cells $\times$ 56 method $\times$ 2 modes 의 매트릭스에서 비교 실험군에 해당하는 method 는 9 cells $\times$ 2 modes 모두 완료했지만, 미커버 영역을 9 가지 카테고리 정직하게 분류한다. 가장 큰 비중은 자원 한계 (birch 메모리 50-200GB 폭증), 알고리즘 정독 검토 (5월 10일 8-agent code audit 으로 발견한 reference 위반 23 method 폐기), 그리고 본 논문 §V-A multi-table 영역과의 경계 (우리는 §V-B 영역에 한정)다.

본 연구는 산술 평균이라는 가장 단순한 결합 방식부터 측정을 시작하여 결합 framework 자체의 효과를 baseline 수준에서 입증하였다. 실제 산업 적용을 위해서는 데이터셋의 분포 특성, 차원, 질의 선택도에 따라 결합 방식이 동적으로 결정되는 data-aware ensemble framework 가 필요하다. 본 연구의 산술 평균 결과는 그 framework 의 출발점이며, 향후 연구는 두 그룹으로 나뉜다.

Group A — Data-aware ensemble framework 5 방향: (A1) Distribution-aware ensemble — skew / dense 데이터셋에 따라 결합 가중치 동적 결정, (A2) Dimensionality-aware ensemble — 차원 별 결합 방식 선택, (A3) Estimator-confidence-aware — 각 estimator 의 분산 추정 기반 분산 최소화 가중치, (A4) Query-aware ensemble — 선택도 별 결합 방식 변경, (A5) Meta-learning adaptive ensemble — 측정 환경 특성으로 결합 가중치 학습.

Group B — 일반 확장 5 방향: (B1) 다른 데이터셋 일반화 — YFCC, GLOVE 등에서 동일 패턴 검증, (B2) 이론적 분산 분해 — Cochran 1977 §11.10 composite estimator 적용, (B3) 논문 동적 framework 와 완전 정합 — Q-error 신호 source 명시, (B4) 실제 시스템 적용 — pgvector 또는 Exqutor prototype 통합, (B5) 다른 결합 방식 추가 비교 — 본 연구의 가중치 sweep + 4 cheap 근사 후보 외 다른 결합 방식.

가까운 시일의 일정은 두 가지다. 5월 28일 임채림 박사님과의 SAP 미팅에서 본 연구 결과를 추가 검증한다. 6월 11일 최종 보고서에서 위 두 그룹의 향후 방향 중에서 다음 분기에 우선순위로 둘 영역을 명시한다.

부록 — Method 메커니즘 분석 (Q&A 참조용)

이 부록은 발표 본문에서 한 줄로만 짚은 method-level consistency 와 3-axis sensitivity 분석 패턴을 풀어 둔 것이다. 청자가 Q&A 에서 묻거나, 박광현 교수님 / 임채림 박사님 같은 도메인 전문가가 본 연구의 method 분류 axis 에 대해 질문할 경우의 참조용이다.

부록 A — Method-level consistency

본 연구의 결과를 paradigm 단위로만 보면 paradigm 우위를 단정 짓기 어렵다. paradigm 안에 wavelet_hist (클러스터링, +68% 더 나빠짐), lp_bound (차원 축소, +16% 더 나빠짐) 같은 극단값 method 가 평균을 끌어 올리거나 끌어 내리기 때문이다. 진짜 finding 은 12 개의 anchor method 가 다양한 환경 (데이터셋, cell, 선택도) 에서 모두 비슷한 정도로 (cell 전반에서 -9 ~ -10%) 일관되게 개선한다는 것이다. paradigm 분류 자체보다는 method 안의 내부 sampling 메커니즘 — 클러스터 quality 에 얼마나 의존하는지, hash 기반인지 space-filling curve 기반인지 — 이 더 본질적인 분류 기준이라는 시사점이다.

부록 B — 3-axis sensitivity 분석 (2 axis 일치 + 1 axis 다른 분류)

5월 12 ~ 13 일에 진행한 세 가지 추가 측정 영역에서 method 별 민감도 패턴을 측정했다.

(1) **K granularity 민감도**: 클러스터 개수 K 를 10, 20, 30 으로 바꾼 측정. sparse_rp 는 K=20 에서 sweet spot 의 U 모양 (K-sensitive). chao_weighted 는 K=20 sweet spot 패턴. 반면 hilbert_real 과 hyperloglog 두 method 는 K 값에 거의 영향을 받지 않는다 (K-robust).

(2) **multi-join 재학습 민감도**: 두 벡터 테이블을 864 차원으로 합쳐서 KM20 재학습한 8 measurement. sparse_rp 와 chao_weighted 는 CaseA 모드에서 추가 개선 (multi-jn sensitive), hilbert_real 과 hyperloglog 는 거의 차이 없음 (robust). 결합 모드에서는 4 method 모두 차이 없음.

(3) **Centroid tuple cheap 근사 친화도**: 비싼 multi-join 재학습 대신 single-table 클러스터의 tuple folding 으로 cheap 근사. 결합 모드에서 4 method 모두 평균 -0.84%p 추가 정확도이지만, 친화도 분류는 method 별로 다르다 — hyperloglog 와 chao_weighted 가 가장 큰 추가 개선 (Friendly), sparse_rp 는 중간 (Indifferent), hilbert_real 은 CaseA 단독 대체에서 harmful (Hostile).

3-axis 분류 매트릭스:

Method	K granularity	Multi-jn	Cheap 근사 친화도
sparse_rp	K-sensitive (U-shape)	sensitive	Indifferent
chao_weighted	K=20 sweet	sensitive	Friendly
hilbert_real	K-robust	robust	Hostile (CaseA harmful)
hyperloglog	K-robust	robust	Friendly

본 연구는 K granularity 변화와 multi-table 재계층화 두 측정에서 method 별 민감도 패턴이 일치 (sparse_rp + chao_weighted sensitive vs hilbert_real + hyperloglog robust) 함을 확인하였다. 그러나 저비용 근사 친화도는 다른 분류 패턴 (Friendly: hyperloglog + chao_weighted) 을 보였다. 즉 method 의 내부 메커니즘이 측정 영역에 따라 다른 영향을 미친다. 2 axis 일치는 본 연구의 method-level consistency 의 evidence 이지만, 1 axis 다른 분류는 method 의 메커니즘 차이가 단순 sensitivity 분류로 환원되지 않음을 시사한다.

부록 H — 폐기 method 전체 list (박세은 5/13 12:13 피드백 반영)

본문에 핵심 method 만 짚고 폐기 method 명 전체 list 는 이 부록으로 분리.

자원 한계 폐기 6 종: birch, agglomerative, hdbscan, kde_parzen, dirichlet, kernelpca, neuocard.

알고리즘 구현 결함 폐기 23 종: 5월 10일 8-agent code audit 발견. 주요 사례 — kdtree (`idx % n_strata` 와 등가), vinecopula (rank+PCA1D 별칭), neuram (PCA1D 100% 동일), ams_count_sketch (lsh 와 한 줄씩 동일) 등.

정합성 위반 폐기 9 종: halton, sobol, lhs, hammersley, dense_rp, random_projection, dbscan, ccsketch, lsh, ams_count_sketch.

Q&A 예상 질문 대비

발표 후 Q&A 에서 나올 수 있는 질문을 미리 정리해 둔다. 답변은 모두 정직하게 한계를 인정하고 측정 결과로 답하는 톤이다.

Q1. 왜 결함 모드의 산술 평균이 효과적인가? 이론적으로 어떤 근거가 있는가? 베르누이 무작위 샘플링은 편향이 0 이지만 분산이 크다. 클러스터 기반 계층적 표집 추정량은 분포 가정이 맞을 때는 분산이 작지만, 가정이 빗나가면 편향이 생길 수 있다. 두 추정값을 산술 평균하면 한 쪽이 실패할 때 다른 쪽이 보완해 주는 안정적인 구조가 된다. 측정 결과로는 92.5% 의 cell 에서 단일 베르누이보다 더 정확하다. 가중치 sweep 측정으로 산술 평균이 가중치 변화 중 best 임도 확인됐다. 다만 결함으로 단독 best 능가는 불가능했음을 정직하게 짚는다 — 결함 best (-7.37%) < 단독 best (-10.17%).

Q2. 단독 대체의 효과는 어떻게 보아야 하는가? 56 방법 중 약 40% 가 평균적으로는 베르누이 기준선보다 정확했다. 통계 검정으로 cell 전반에서 안정적으로 베르누이를 우위로 누른 method 는 7.6% 정도였다. 이 15 method 의 평균 개선폭은 -5 ~ -12% 인데, paper 자체 재현 시 발생하는 측정 변동 (-4.3%) 보다 1.2 ~ 3 배 큰 의미 있는 개선이다. 본 연구의 main finding 이며 단독 best 인 minibatch_partial 은 -10.17% 개선이다. 다만 cell 별 spread 가 커서 산업 적용 안정성 부족하므로 결함 framework 가 보조 역할로 가치 있다.

Q3. paradigm 평균이 극단값에 의해 영향을 받는다면, 우리 결과의 정직성은 어떻게 확보되는가? 이 점은 5월 13 일 새벽에 강재현이 지적한 부분이다. 우리는 paradigm 평균과 함께 12 개의 각 paradigm 대표 방법 (anchor method) 별로 -9 ~ -10% 의 일관된 개선폭과 안정적인 spread 를 보여주는 method 단위 분석을 별도로 제시한다. 즉 "paradigm 우위" 보다 "anchor method 일관성" 이 우리의 진짜 결과다.

Q4. multi-table 영역으로의 확장은 어떻게 가능한가? 8 measurement 의 multi-join 재학습 결과로 method 마다 민감도가 다르다는 것을 확인했다. quality 의존적인 두 method (sparse_rp, chao_weighted) 에는 비싼 재학습이 추가 개선을 주지만, quality 안정적인 두 method (Hilbert curve, HyperLogLog) 에는 차이가 없다. 또한 Centroid tuple cheap 근사가 비싼 재학습보다 결합 모드에서 더 좋은 결과를 주기 때문에 (4 method 모두 평균 -0.84%p 추가 개선), 학습 비용 추가 없이 multi-table 영역으로 확장 가능하다.

Q5. paper 의 결과와 우리 측정값의 차이는? -4.3% 차이는 측정 변동 (measurement variance) 범위 내다. paper 가 절사 평균으로 보고하는데, seed 와 query 집합이 다르면 이 정도 변동은 일반적이다. 같은 hyperparameter 와 같은 query 정의를 사용했고, JSON 산출물 전수 검사에서 결손 없음이 확인된다.

Q6. 산업에 어떻게 적용 가능한가? 세 가지 영역. (1) 영역 A 일반 OLAP server: sparse_rp 또는 chao_weighted 단독 대체 + 산술 평균 결합 보조. (2) 영역 B 정확도 우선: neuram 단독 대체. (3) ★ 영역 C 모바일/embedded: **reservoir 단독 대체 — 메모리 O(1) + 정확도 anchor 수준**. 본 연구의 가장 강력한 산업 적용 finding. PostgreSQL pgvector 또는 Exqutor §V-B 모듈 통합이 구체 적용 영역.

Q7. 폐기된 method 가 많은데, 본 연구의 결과 신뢰도는 어떻게 보아야 하는가? 폐기 사유가 측정 과정에서 명확하게 드러나는 사유다. 자원 한계 (birch 의 메모리 50-200GB 폭증, agglomerative 의 OOM, kde_parzen 의 4 시간 timeout) 는 측정 서버의 물리적 제약, 알고리즘 정독 검토에서 발견된 reference 위반 (kdtree 의 단순 hash, vinecopula 의 PCA1D 별칭, neuram = PCA1D 100% 동일) 은 학술 정직성 위반, 정합성 위반 9 종은 큰 데이터셋에서 외곽 값이 나오는 안정성 문제다. 폐기를 명시적으로 보고서에 분류한 것이 본 연구의 정직성 axis 다.

마치며

이 발표 시나리오는 4월 28일 중간 보고서를 기준으로 잡고, 5월 한 달의 RQ3 측정과 추가 분석을 7 단계 순차 흐름 (문제 정의 → 분포 인지 방법 56 탐색과 폐기 분류 → 단독 대체 가능성 → 결합 framework → 자원 효율 → 권장 design → 마무리) 으로 발표 자리에서 풀어 가는 정리다. 본 v2 의 핵심 정정은 단독 대체 우선 + 결합 보조 narrative 갱신과 박세은 피드백 반영 (method 개수 줄임 + 숫자/공식 최소화) 이다. 발표 시간이 약 15분이라는 점을 고려해서 도입과 RQ1·RQ2 에 3분, 분포 인지 방법 탐색에 2.5분, 단독 대체 가능성에 2.5분, 결합 framework 에 2분, 자원 효율에 2.5분, 권장 design 에 1.5분, 마무리와 Q&A 에 2분 정도를 배분했다.

다음 단계는 이 흐름대로 슬라이드 20 장을 정리하고, 5월 28일 임채림 박사님 미팅 전에 한 차례 리허설을 진행하는 것이다.

작성: 2026-05-14 KST · v2 4차 정정